1 Background

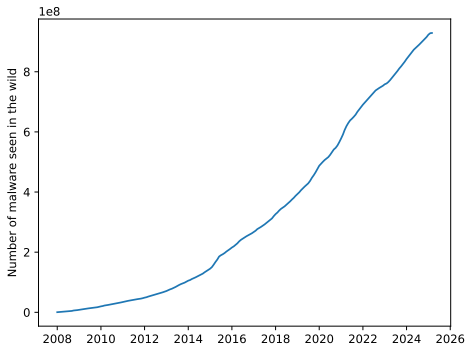

Figure 1: Number of malicious programs seen in the wild since 2008, data taken from [8]

Malicious software, otherwise known as malware, are widely considered to be one of the most pervasive dangers faced by users in the current digital threat landscape [6]. In recent years, the prevelance of this form of software has been on the increase, with nearly 100 million occurances seen since the beginning of 2024 alone (Figure 1). Mitigation of the risks posed by these programs is thus of the utmost importance to security researchers, and has been an active area of research in the field [15].

In this literature review, we explore the defining characteristics of malware, how threat actors evade detection by existing anti-malware services, and a number of proposed approaches that leverage machine learning to more reliably detect these programs.

1.1 Malware

According to Kramer and Bradfield [11], malware are defined as programs that cause other software to deviate from its intended behavior in a harmful way. More generally, they can be thought of as programs that perform detrimental, or unwanted actions on a host system, without the knowledge of its user. They are one of the most powerful tools available to threat actors, and are often designed to be as covert as possible in order to maximise the time before which their actions are noticed. This makes them a pervasive threat to digital infrastructure across the globe. For example, in 2017 the WannaCry ransomware infected the National Health Service, and led to the institution being unable to access critical healthcare infrastructure [16] for the attacks duration.

Security researchers often categorise these programs based on how they spread between hosts, and their actions on a host system. It is worth noting that categories within the following taxonomy are not mutually exclusive, as modern malware often exist in multiple classes at once.

- **Worm** — Malicious software that automatically propagate throughout a network.
- **Virus** — Programs that modify other programs to replicate itself throughout an infected host.
- **Trojans** — Malware that disguises itself as legitimate software to infect a host.
- **Backdoor** — Software that provide covert remote access to infected systems.
- **Ransomware** — Malware that deny access to systems or information until a fee is paid.
- **Information Stealer** — Malware that extract information, such as credentials and user activity
- **Botnet** — Often part of a larger network of infected systems, controlled by a threat actor to perform large scale attacks against digital infrastructure.
- **Rootkit** — Malware providing privileged access to an infected system.

1.2 Obfuscation

```
rule ExampleRule
{
    strings:
        $a = "malicious string"
        $b = { 6A 40 68 00 30 00 00 }

    condition:
        $a or $b
}
```

Listing 1: An example YARA rule

One common approach to malware detection often used by commercial services make use of a set of rules that define sequences of bytes, found in known malicious programs. If an unknown program were to match with any of these rules, it can be flagged as malicious, and we have provided an example of such a rule in listing 1. A large collection of these rules are maintained by a community of security researchers, and provide an efficient and effective means by which to detect frequently seen malware.

To avoid detection by these rules, malware developers will often modify their programs in a technique known as obfuscation [25] whereby characteristics of a program that may be indicative of its behavior are obscured. This makes understanding a program without executing it challenging to both human security researchers and automated analysis tools alike. As a result of this, most modern malware often feature some form of obfuscation. However, it is worth noting that it is not exclusive to malicious programs, as commercial software developers may also employ these techniques to protect their intellectual property. This is not common practise however,

and a majority of benign programs do not feature these techniques. As a result of this, the datasets used to train programs

Many forms of obfuscation exist, and new techniques are observed in the wild by security researchers frequently.

1.3 Machine Learning

1.4 Detection

A number of approaches to malware detection have been proposed in the literature. These largely fall into three categories, based on how features are extracted for later use for classification, these are as follows:

- **Static** — Features extracted from the structure of a program, including metadata, instructions, and data found within the executable.
- **Dynamic** — Features extracted during the execution of the program, including external function calls, communication, and modifications made to the host.
- **Hybrid** — A combination of both *static* and *dynamic* features

1.4.1 Static

Static detection methods leverage features derived from the programs structure rather than its behaviour. We have seen a number of static features used to detect malware in the literature relating to this topic, such as instruction sequences [1, 21], strings [23], external functions [22], and the executable header [20].

In [22], the authors propose using static features such as printable strings, and external API calls to train conventional machine learning algoriths for malware detection. In this investigation, the authors found that the naive bayes algorithm, trained on an executables strings gave the best performance, with 97% classification accuracy. In addition to this, they also tested an inductive rule learning algorithm known as RIPPER. This algorithm was trained on information pertaining to what external functions and libraries are used by any given program. They found that this approach was sub-optimal, accurately classifying programs only approximately 90% of the time, that we believe to be a result of the high degree of overlap between external methods used by malicious and benign programs, albeit for different purposes. Furthermore, in their experiments, the authors used a small unblanced dataset of only approximately 4000 samples, of which over two-thirds were malicious, which would likely have an adverse effect on the generalisability of their proposed models.

A great deal of previous research into static malware detection utilise Convolutional Neural Networks (CNNs) [24, 17, 9, 15]. These approaches firstly convert programs to grayscale images, upon which a CNN is then trained on. They have been shown to be a promising approach to this problem, as CNNs are effective at learning a wide range of classification problems [7, 12]. However, as CNNs expect inputs of fixed size, it becomes necessary to resize the images to a fixed resolution before they can be trained or infered upon. Image resizing is a lossy operation, whereby important information may be lost, meaning that over datasets where there is significant variation in executable size, the trained models may struggle to accurately identify malicious programs.

Instruction sequences have also been explored as static features in malware detection. For example, in [1, 21], the authors utilise hidden markov models and achieve promising results. As these models consider what actions are being performed by the host system without having to execute the program, they offer a solution that we believe to be more resiliant to obfuscation than other static approaches.

Extending upon the idea of utilising instruction sequences for malware detection, Demırcı et al. [3] outline an approach in which they propose the use of stacked bidirectional Long-Short Term Memory (BiLSTM) and pre-trained language models including BERT [4], and GPT-2 [18]. In their investigation, the BiLSTM model was able to achieve 98% classification accuracy whereas the pre-trained transformer models achieved only 77%. Transformer models have been shown to be a capable method in sequence classification problems, and as such, this result is surprising. We believe this to be a result of the corpora used to pre-train the transformer models used in their investigation, which are comprised of mostly english texts. The understanding of these models would thus be limited to langauges similar to english, and may struggle to understand the introcases of assembly langauge, impacting their ability to reliably classify malware.

Previous research has also established the use of pre-trained language models in malware detection on the metadata of an executable. In [19], Rahali and Akhloufi fine-tune BERT on text gathered from the manifest file found in android applications. This file contains information pertaining to the activities, background tasks, event listeners, and permissions granted to an application. Their model was able to achieve 97% binary classification accuracy, in comparison to [3] which was only able to achieve 77% with the same model, trained on instruction sequences. We believe this discrepancy to be a result of the language found in these manifest files being more closely related to the pre-training corpora of the BERT model. This would enable their model to better understand the information found in the manifest, and its use in distinguishing between benign and malicious executables. Their approach however, is limited to environments where a file containing metadata are required,

such as mobile operating systems, making it unsuitable for desktop malware detection.

1.4.2 Dynamic

In comparison to static methods, dynamic methods consider the behavioral characteristics of a program during its execution. This requires the use of an isolated environment, such as a virtual machine to safely execute a program to observe its behavior [2]. These approaches are known to be more resiliant to obfuscation, at the high cost of complexity and computational resources.

Our research indicates that much of the work in this field leverage API call sequences to distinguish between malicious and benign programs. This enables these approaches to develop an understanding of actions taken by a program on a host system, and what actions may be indicative of a malicious program. This has been shown to be an effective approach to dynamic malware detection. For example, in [10], the authors propose a signature based approach that was shown to be highly accurate in their experiments with an accuracy of approximately 99%. The authors curate a database of subsequences often associated with malicious programs through the use of multi-sequence alignment. However, we do not believe their approach would be resiliant to obfuscation, as a malware developer could simply insert junk api calls in between other important calls to avoid detection by the proposed approach.

In their 2022 paper, Li et al. [13], the authors propose a detection method that also makes use of API call sequence information. However, they extend upon previous research by providing this information in the form of a graph, where shared arguments between two consecuitive api calls are given as edge data. A graph neural network is then trained to classify this information, and was able to achieve 98% accuracy over a large dataset of 40,000 samples. In comparison to [10], junk API calls would be disconnected in the graph, and the model could easily learn to ignore any such elements.

However, some forms of malware obfuscate their API calls, effectively defeating these approaches [14]. As a result of this, other features that are more resiliant to obfuscation have been explored. For example, in [5], the authors outline an approach that again makes use of graph neural networks. However, instead of considering just API calls, they develop a behavioral profile in order to detect highly sophisticated malware, such as those developed by Advanced Persistent Threats (APTs). APTs are often sponsored by a nation state, and thus have access to advanced tools, such as zero-day exploits, and advanced obfuscation techniques, making detection challenging. This behavioral profile is based on system events triggered by a program, and includes information relating to the creation of processes, changes made to the file system and registry, and any network communications. This method is able to reliably detect these sophisticated forms of malware, and is resiliant to advanced forms of obfuscation. However, this comes at a high cost in terms of computational resources.

1.5 Conclusions and Future Research Directions

From our research, we have found that there are a number of approaches to malware detection that utilise a variety of features.

Static approaches, whereby only the structure of an executable is considered rely heavily on image-based representations of programs, which a CNN can then be trained on. The success of these approaches vary, but we found that as a result of variability in the size of an executable, important features may be lost when resizing the image to a fixed resolution.

We have also found that sequential models, that learn to detect malware based on the instructions performed throughout the course of a program execution are a promising area of investigation. The BiLSTM model proposed in [3] achieves high detection accuracy, and also suggest that pre-trained transformer models, such as BERT are less suitable to this task. However, these models have been shown to be well suited to sequence classification, and we believe that they could be a viable approach to malware detection if they were to be pre-trained on assembly instructions rather than natural languages such as english. Other approaches that leverage transformer architectures seem to verify this claim, as seen in [19], where a transformer architecture was fine-tuned to classify malware using the manifest file found in android executables. The contents of this file are more similar to the language used in pre-training, and their model would thus have a better understanding of the file, enhancing its classification accuracy.

Dynamic detection methods on the other hand, despite being more resource intensive, seem to be more capable to detecting heavily obfuscated executables, through the observation of an executables behavior rather than its structure. Much of the work done in this area of the field extract sequences of API calls made during a programs execution, upon which the executable can be classified. This is a powerful approach to malware detection, however significant infrastructure is necessary to safely execute a program in an isolated environment. Furthermore, it is possible to obfuscate API calls, as shown in [14] that degrade the effectiveness of this approach. While it is possible to develop behavioral profiles to aid these approaches resiliance to obfuscation, this comes at a high cost in computation.

We believe that by pre-training our own transformer based model, such as BERT, on assembly instructions, and then fine tuning this model to classify programs, we can achieve a good balance between computational efficiency, and detection accuracy. We will also investigate

this models resiliance to obfuscation, by obfuscating our benign programs to varying degrees.

References

[1] Srilatha Attaluri, Scott McGhee, and Mark Stamp. "Profile Hidden Markov Models and Metamorphic Virus Detection". In: *Journal in Computer Virology* 5.2 (May 1, 2009), pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. URL: https://doi.org/10.1007/s11416-008-0105-1 (visited on 02/25/2025).

[2] Marcus Botacin et al. "Challenges and Pitfalls in Malware Research". In: *Computers & Security* 106 (July 1, 2021), p. 102287. ISSN: 0167-4048. DOI: 10.1016/j.cose.2021.102287. URL: https://www.sciencedirect.com/science/article/pii/S0167404821001115 (visited on 12/05/2024).

[3] Denız Demırcı et al. "Static Malware Detection Using Stacked BiLSTM and GPT-2". In: *IEEE Access* 10 (2022), pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. URL: https://ieeexplore.ieee.org/document/9785789?utm_source=chatgpt.com (visited on 02/25/2025).

[4] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. May 24, 2019. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. URL: http://arxiv.org/abs/1810.04805 (visited on 02/28/2025). Pre-published.

[5] Cho Do Xuan and given-i=DT family=Huong given=DT. "A New Approach for APT Malware Detection Based on Deep Graph Network for Endpoint Systems". In: *Applied Intelligence* 52.12 (Sept. 1, 2022), pp. 14005–14024. ISSN: 1573-7497. DOI: 10.1007/s10489-021-03138-z. URL: https://doi.org/10.1007/s10489-021-03138-z (visited on 02/18/2025).

[6] European Union Agency for Cybersecurity. *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office, 2024. URL: https://data.europa.eu/doi/10.2824/0710888 (visited on 02/21/2025).

[7] Omar Habibi, Mohammed Chemmakha, and Mohamed Lazaar. "Performance Evaluation of CNN and Pre-trained Models for Malware Classification". In: *Arabian Journal for Science and Engineering* 48.8 (Aug. 1, 2023), pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. URL: https://doi.org/10.1007/s13369-023-07608-z (visited on 02/25/2025).

[8] AV-TEST-The Independent IT-Security Institute. *AV-ATLAS - Malware & PUA*. AV-ATLAS - Malware & PUA. URL: https://portal.av-atlas.org/malware (visited on 12/19/2024).

[9] M. Jain, W. Andreopoulos, and M. Stamp. "Convolutional Neural Networks and Extreme Learning Machines for Malware Classification". In: *Journal of Computer Virology and Hacking Techniques* 16.3 (2020), pp. 229–244. DOI: 10.1007/s11416-020-00354-y.

[10] Youngjoon Ki, Eunjin Kim, and Huy Kang Kim. "A Novel Approach to Detect Malware Based on API Call Sequence Analysis". In: *International Journal of Distributed Sensor Networks* 11.6 (June 1, 2015), p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. URL: https://doi.org/10.1155/2015/659101 (visited on 11/02/2024).

[11] Simon Kramer and Julian C. Bradfield. "A General Definition of Malware". In: *Journal in Computer Virology* 6.2 (May 1, 2010), pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. URL: https://doi.org/10.1007/s11416-009-0137-1 (visited on 02/21/2025).

[12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Communications of the ACM* 60.6 (May 24, 2017), pp. 84–90. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3065386. URL: https://dl.acm.org/doi/10.1145/3065386 (visited on 02/28/2025).

[13] Ce Li et al. "DMalNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning". In: *Computers & Security* 122 (Nov. 1, 2022), p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. URL: https://www.sciencedirect.com/science/article/pii/S0167404822002668 (visited on 11/27/2024).

[14] Yang Li et al. "APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity". In: *Applied Sciences* 13.16 (16 Jan. 2023), p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. URL: https://www.mdpi.com/2076-3417/13/16/9056 (visited on 02/25/2025).

[15] Pascal Maniriho, Abdun Naser Mahmood, and Mohammad Jabed Morshed Chowdhury. "A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions". In: *Journal of Systems and Software* 209 (Mar. 1, 2024), p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. URL: https://www.sciencedirect.com/science/article/pii/S0164121223003163 (visited on 02/18/2025).

[16] National Audit Office (NAO). *Investigation: WannaCry Cyber Attack and the NHS*. London, England: National Audit Office, Oct. 2017. URL: https://web.archive.org/web/20241212140331/https://www.nao.org.uk/reports/investigation-wannacry-cyber-attack-and-the-nhs/.

[17] Sang Ni, Quan Qian, and Rui Zhang. "Malware Identification Using Visualization Images and Deep Learning". In: *Computers & Security* 77 (Aug. 1, 2018), pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/j.cose.2018.04.005. URL: https://www.sciencedirect.com/science/article/pii/S0167404818303481 (visited on 02/21/2025).

[18] Alec Radford et al. "Language Models Are Unsupervised Multitask Learners". In: ().

[19] Abir Rahali and Moulay A. Akhloufi. "MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers". In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Oct. 2021, pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. URL: https://ieeexplore.ieee.org/abstract/document/9659287 (visited on 02/25/2025).

[20] Vinayakumar Ravi et al. "A Multi-View Attention-Based Deep Learning Framework for Malware Detection in Smart Healthcare Systems". In: *Computer Communications* 195 (Nov. 1, 2022), pp. 73–81. ISSN: 0140-3664. DOI: 10.1016/j.comcom.2022.08.015. URL: https://www.sciencedirect.com/science/article/pii/S0140366422003231 (visited on 02/25/2025).

[21] Neha Runwal, Richard M. Low, and Mark Stamp. "Opcode Graph Similarity and Metamorphic Detection". In: *Journal in Computer Virology* 8.1 (May 1, 2012), pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. URL: https://doi.org/10.1007/s11416-012-0160-5 (visited on 02/25/2025).

[22] M.G. Schultz et al. "Data Mining Methods for Detection of New Malicious Executables". In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001. May 2001, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. URL: https://ieeexplore.ieee.org/document/924286 (visited on 02/21/2025).

[23] P. V. Shijo and A. Salim. "Integrated Static and Dynamic Analysis for Malware Detection". In: *Procedia Computer Science*. Proceedings of the International Conference on Information and Communication Technologies, ICICT 2014, 3-5 December 2014 at Bolgatty Palace & Island Resort, Kochi, India 46 (Jan. 1, 2015), pp. 804–811. ISSN: 1877-0509. DOI: 10.1016/j.procs.2015.02.149. URL: https://www.sciencedirect.com/science/article/pii/S1877050915002136 (visited on 02/25/2025).

[24] Jiawei Su et al. "Lightweight Classification of IoT Malware Based on Image Recognition". In: *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*. Vol. 02. 2018, pp. 664–669. DOI: 10.1109/COMPSAC.2018.10315.

[25] Ilsun You and Kangbin Yim. "Malware Obfuscation Techniques: A Brief Survey". In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*. 2010, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.